

Career & Job Advisor

Team Members:

Shivesh Saravanan	1002211909
Varun Sambath Kumar	1002230296
Nantha Kumar Ashok Anand	1002224704

Project Description:

The Career & Job Advisor is a career management platform designed to support individuals in their job-seeking and professional development journey. It combines essential job search and application tracking tools with intelligent, AI-powered insights to provide a centralized solution for career planning.

The platform enables users to search for jobs by title, location, or industry, manage and upload resumes, maintain career profiles, and track application progress. In addition, users receive notifications about new opportunities, schedule and monitor interviews, and access curated resources to enhance their employability. The system goes beyond traditional job portals by integrating AI features, including skill gap analysis and career resource recommendations, ensuring users are better prepared for competitive job markets.

Project Scope:

This project aims to enhance the job search and career planning process by allowing users to search for jobs, upload resumes, manage profiles, and track job applications seamlessly. Users can also schedule interviews, receive real-time notifications for job postings, and access AI-driven insights such as skill gap analysis and career resources.

The scope of the project includes delivering all the core features within the development cycle to ensure a smooth, efficient, and user-friendly career management experience.

Deliverable Functionalities:

For Job Seekers (Users)

- Job Search – Search for jobs by title, location, or industry using integrated job APIs.
- Resume Upload & Storage – Upload and store resumes securely for easy access during applications.

- Profile Management – Create and maintain personal profiles, including skills, experiences, and career goals.
- Application Tracking – Track applied jobs, monitor status updates, and maintain a history of applications.
- Notifications & Alerts – Receive real-time alerts for new job postings, application updates, and interview reminders.
- Interview Scheduling – Book and manage interviews directly through the platform with integrated scheduling tools.

AI Features

- Skill Gap Analysis – Analyze uploaded resumes and job descriptions to suggest skills users should develop to qualify for desired jobs.
- Career Resources – Provide curated articles, training courses, and professional advice tailored to the user's career path.

Project Limitations & Assumptions:

- Mobile app development is out of scope (only web-based application)
- Third-party calendar synchronization (Google Calendar, Outlook) will not be included in the first release.
- Video conferencing for interviews is out of scope (only scheduling is supported).
- Offline job application processes are not supported (only online application tracking is covered).

Project Resources:

To successfully implement the Career & Job Advisor system, the following resources will be utilized. As this project is carried out by a team of three members, responsibilities are divided to balance development, design, and AI integration effectively.

1. Technology Stack

- Frontend: Next.js, React, TailwindCSS
- Backend: Node.js with Express
- Database: PostgreSQL

- AI & ML Services:
 - OpenAI API for skill gap analysis
 - OpenAI APIs / NLP libraries for resume parsing & career suggestions
- Authentication & Security:
 - JWT-based authentication
 - bcrypt for password hashing
 - HTTPS/SSL encryption
- Notifications & Scheduling:
 - WebSockets Messaging for notifications
 - Calendar module for interview scheduling
- APIs Integration: Greenhouse / Lever job APIs for job postings

2. Infrastructure & Hosting – Docker

3. Development & Collaboration Tools

- Code Development: Visual Studio Code (VS Code)
- Version Control: Git & GitHub (repository + branching model)
- Project Management: Notion
- Testing: Jest, Vitest and Selenium

Development Methodology Agile (Scrum):

The Career & Job Advisor Web Application follows the Agile Development Methodology (Scrum). Agile ensures flexibility, incremental improvements, and continuous feedback integration throughout the development cycle.

Why we choose Scrum?

- **Short timeline (7 weeks):** Scrum works in short, time-boxed iterations (sprints), which aligns perfectly with our 7-week plan.
- **Small team:** Scrum is lightweight, with clearly defined roles (Product Owner, Scrum Master, Developers), making it ideal for small teams.

- **Evolving requirements:** Features like AI-based skill gap analysis, job search integration, and notifications may need refinement after early testing. Scrum allows scope adjustment at sprint boundaries.
- **Improved Collaboration:** The Scrum framework enhances team coordination, transparency, and task prioritization through daily stand-up meetings.
- **Continuous feedback:** Each sprint delivers a working increment (e.g., job search, resume upload, AI features), enabling user feedback before final deployment.
- **Risk management:** Technical risks such as AI accuracy, API downtime, and security concerns can be identified and resolved early through sprint reviews and retrospectives.

Sprint Backlog:

Sprint 1 (22 Sep – 28 Sep 2025) – Project Setup & Requirement Analysis

Task Category	Tasks
Requirement Analysis	Define user stories and acceptance criteria
Project Setup	Create GitHub repo, setup CI/CD pipeline
Environment Setup	Configure Docker Compose (web, API, DB, AI)
Database Design	Draft PostgreSQL schema (Users, Resumes, Applications)

Sprint 2 (29 Sep – 5 Oct 2025) – Database & Authentication

Task Category	Tasks
Database Schema	Implement tables for Users, Roles, Resumes
Authentication	Build JWT-based login/signup
Session Management	Configure user sessions
Testing	Write unit tests for authentication APIs

Sprint 3 (6 Oct –12 Oct 2025) – Frontend Development (UI/UX)

Task Category	Tasks
Dashboard UI	Develop job seeker dashboard (Next.js + Tailwind)
Resume Upload UI	Build resume upload form (PDF/DOCX validation)
Profile Management UI	Create pages for skills, experience, goals
API Integration	Connect frontend to authentication API

Sprint 4 (13 Oct –19 Oct 2025) – Backend APIs (Job Search & Resume Handling)

Task Category	Tasks
Job Search API	Integrate Greenhouse/Lever job APIs
Resume Handling	Implement resume upload & secure storage
Profile Management	CRUD APIs for user profiles
Application Tracking	Build APIs for job application status updates

Sprint 5 (20 Oct –26 Oct 2025) – AI Features (Skill Gap Analysis & Career Resources)

Task Category	Tasks
AI Service Setup	Build FastAPI microservice for skill gap analysis
Skill Extraction	Compare resume vs. job descriptions (NLP)
Recommendations	Suggest skills and resources (courses/articles)
Validation	Test AI outputs with sample resumes/JDs

Sprint 6 (27 Oct – 2 Nov 2025) – Notifications & Interview Scheduling

Task Category	Tasks
Notifications	Implement WebSockets/Firebase alerts
Email Alerts	Send application/job update notifications
Scheduling	Develop calendar-based interview booking
Reminders	Configure reminders for scheduled interviews

Sprint 7 (3 Nov –11 Nov 2025) – Testing, Deployment & Documentation

Task Category	Tasks
Testing	Perform unit, integration, and UI tests
Security	Test against SQL injection, XSS, CSRF
Deployment	Deploy using Docker Compose (frontend, backend, AI, DB)
Documentation	Prepare user manual & developer guide
Final Demo	Present application to stakeholders

Project Cost Estimation (COCOMO II Model):

For estimating the software size in terms of Source Lines of Code (SLOC), I referred to the GitHub repository AI-Powered Job Portal. This project is similar to our project “Career and Job Advisor with AI”, and it provided a realistic reference point for approximating the amount of code required. Using this as a baseline, the COCOMO (Constructive Cost Model) was applied for software cost estimation and project planning. COCOMO helps in predicting the effort, development time, and overall cost by considering project size, complexity, and other influencing factors.

COCOMO II - Constructive Cost Model

Software Size Sizing Method Source Lines of Code

[SLOC](#)

	% Design Modified	% Code Modified	% Integration Required	Assessment and Assimilation (0% - 8%)	Software Understanding (0% - 50%)	Unfamiliarity (0-1)
New	4000					
Reused		0	0			
Modified						

Software Scale Drivers

Precedentedness	Low	Architecture / Risk Resolution	Nominal	Process Maturity	Nominal
Development Flexibility	High	Team Cohesion	High		

Software Cost Drivers

Product		Personnel		Platform	
Required Software Reliability	Nominal	Analyst Capability	High	Time Constraint	Nominal
Data Base Size	Nominal	Programmer Capability	High	Storage Constraint	Nominal
Product Complexity	High	Personnel Continuity	Nominal	Platform Volatility	Nominal
Developed for Reusability	Nominal	Application Experience	High	Project	
Documentation Match to Lifecycle Needs	Nominal	Platform Experience	Nominal	Use of Software Tools	High
		Language and Toolset Experience	High	Multisite Development	Low
				Required Development Schedule	Nominal

Maintenance Off

Software Labor Rates

Cost per Person-Month (Dollars) 7000

Calculate

Results

Software Development (Elaboration and Construction)

Staffing Profile

Effort = 9.2 Person-months

Schedule = 7.4 Months

Cost = \$64205

Your project is too small to display a staffing profile due to truncation.

Total Equivalent Size = 4000 SLOC

Effort Adjustment Factor (EAF) = 0.69

Acquisition Phase Distribution

Phase	Effort (Person-months)	Schedule (Months)	Average Staff	Cost (Dollars)
Inception	0.6	0.9	0.6	\$3852
Elaboration	2.2	2.8	0.8	\$15409
Construction	7.0	4.6	1.5	\$48796
Transition	1.1	0.9	1.2	\$7705

Software Effort Distribution for RUP/MBASE (Person-Months)

Phase/Activity	Inception	Elaboration	Construction	Transition
Management	0.1	0.3	0.7	0.2
Environment/CM	0.1	0.2	0.3	0.1
Requirements	0.2	0.4	0.6	0.0
Design	0.1	0.8	1.1	0.0
Implementation	0.0	0.3	2.4	0.2
Assessment	0.0	0.2	1.7	0.3
Deployment	0.0	0.1	0.2	0.3

Your output file is at http://softwarecost.org/tools/COCOMO/data/COCOMO_September_26_2025_22_04_01_645620.txt

Created by Ray Madachy at the Naval Postgraduate School. For more information contact him at rjmadach@nps.edu.

Project Timeline:

The project follows an **iterative sprint-based development model (Scrum)**, starting from **September 22, 2025, to November 11, 2025**. Each sprint spans one week and is dedicated to delivering specific functionalities such as authentication, job search, resume upload, AI-based skill gap analysis, notifications, and interview scheduling. This approach ensures **continuous progress, regular testing, and incremental deployment**, allowing the team to refine features and incorporate feedback throughout the development cycle.

Sprint	Start Date	End Date
Sprint 1	Sep 22, 2025	Sep 28, 2025
Sprint 2	Sep 29, 2025	Oct 5, 2025
Sprint 3	Oct 6, 2025	Oct 12, 2025
Sprint 4	Oct 13, 2025	Oct 19, 2025
Sprint 5	Oct 20, 2025	Oct 26, 2025
Sprint 6	Oct 27, 2025	Nov 2, 2025
Sprint 7	Nov 3, 2025	Nov 11, 2025

Project Planning:

Work Breakdown Structure (WBS):

Level 1: Main Phase

1. Project Initiation & Planning
2. Database & Authentication
3. Frontend Development (UI/UX)
4. Backend Development (Job Search, Resume, Profile)
5. AI Features (Skill Gap Analysis & Career Resources)
6. Notifications & Interview Scheduling
7. Testing & Debugging
8. Deployment & Documentation

Level 2: Tasks

1. Project Initiation & Planning

- Define user stories & acceptance criteria.
- Set up GitHub repository.
- Configure Docker Compose environment.
- Initial project plan & sprint allocation.

2. Database & Authentication

- Design PostgreSQL schema (Users, Resumes, Applications).
- Implement JWT-based login/signup.
- Build session management APIs.
- Write unit tests for authentication.

3. Frontend Development (UI/UX)

- Design homepage, login/signup UI & navigation.
- Develop dashboard for job seekers.
- Create resume upload UI (PDF/DOCX validation).
- Build profile management UI (skills, career goals).

4. Backend Development (Job Search, Resume, Profile)

- Integrate job search APIs.
- Implement resume upload & secure storage.
- CRUD APIs for user profile management.
- Application tracking APIs.

5. AI Features (Skill Gap Analysis & Career Resources)

- Build FastAPI microservice for skill extraction & gap analysis.
- Compare resumes with job descriptions using NLP.
- Recommend missing skills and resources.
- Validate outputs with test cases.

6. Notifications & Interview Scheduling

- Implement WebSockets/Firebase for real-time notifications.
- Email alerts for job updates & application status.
- Develop calendar-based interview scheduling.
- Configure reminders for interviews.

7. Testing & Debugging

- Unit testing for backend & AI services.
- Integration testing of APIs with frontend.
- UI/UX usability testing with sample users.
- Security testing (SQL injection, XSS, CSRF).

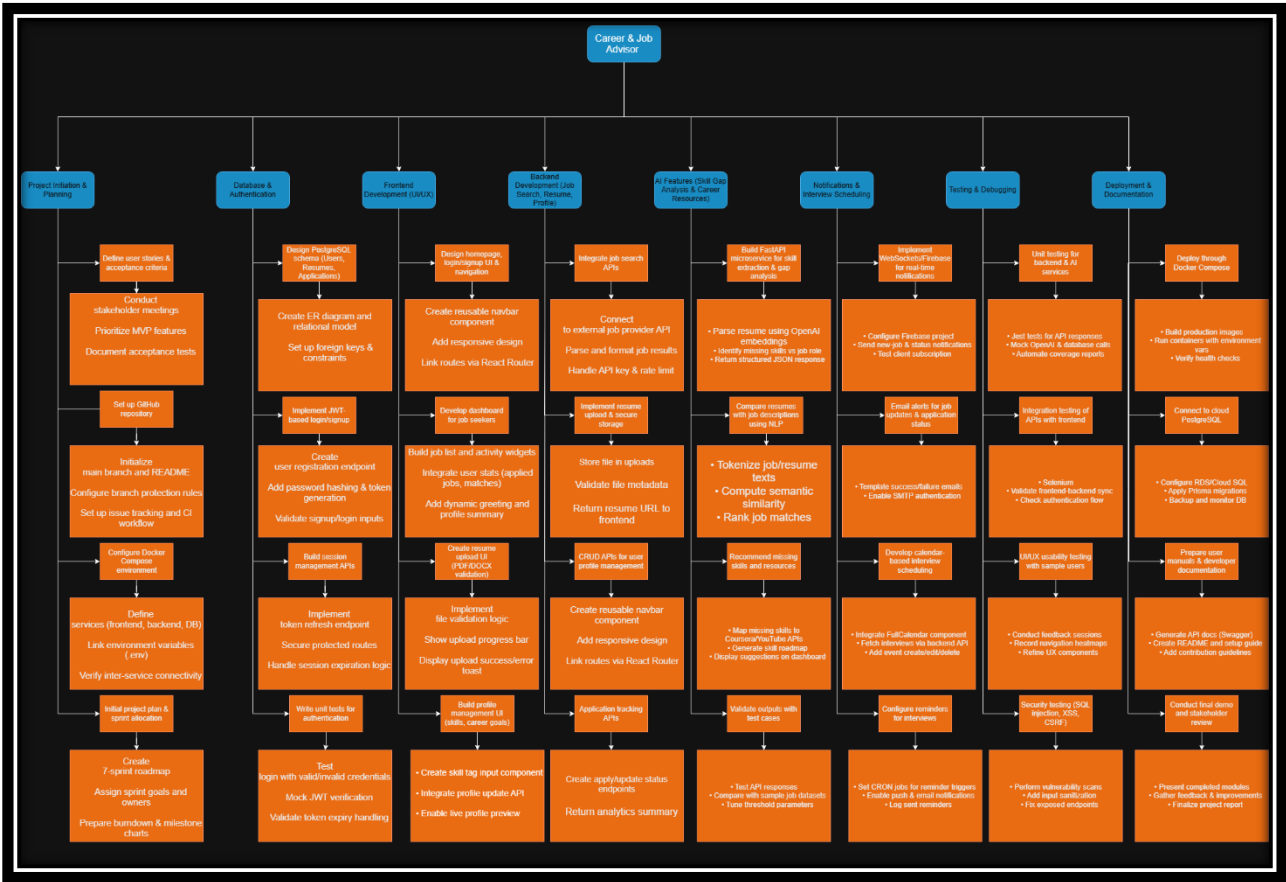
8. Deployment & Documentation

- Deploy through Docker Compose.
- Connect to cloud PostgreSQL.
- Prepare user manuals & developer documentation.
- Conduct final demo and stakeholder review.

Level 3 – Subtasks:

- For **Resume Upload**: Validate file formats, restrict file size, scan for malware.
- For **Skill Gap Analysis**: Extract skills with NLP, compare with job posting requirements, generate suggestions.

- For **Notifications**: Push job posting alerts, send interview reminders.
- For **Testing**: Use Jest (API), PyTest (AI), Selenium (UI automation), Postman (API workflows).

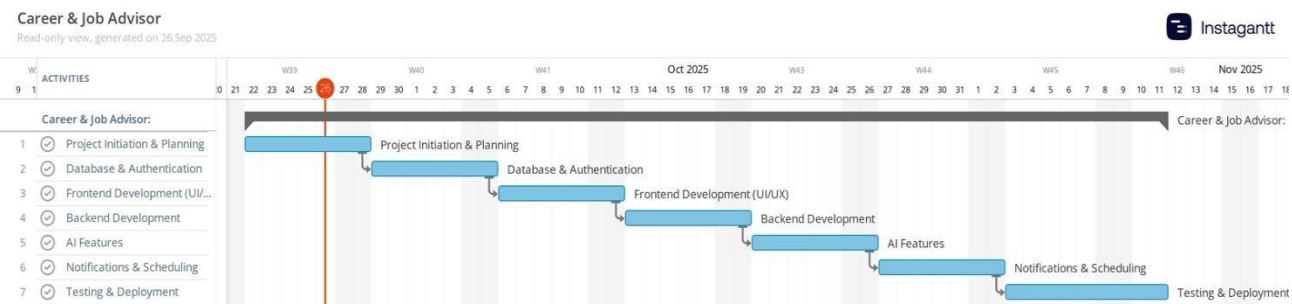


Gantt Chart:

Gantt Chart Timeline

Week	Task	Start Date	End Date	Duration (Days)
Week 1	Project Initiation & Planning	Sep 22, 2025	Sep 28, 2025	7
Week 2	Database & Authentication	Sep 29, 2025	Oct 5, 2025	7
Week 3	Frontend Development (UI/UX)	Oct 6, 2025	Oct 12, 2025	7

Week 4	Backend Development	Oct 13, 2025	Oct 19, 2025	7
Week 5	AI Features	Oct 20, 2025	Oct 26, 2025	7
Week 6	Notifications & Scheduling	Oct 27, 2025	Nov 2, 2025	7
Week 7	Testing & Deployment	Nov 3, 2025	Nov 11, 2025	9



Risk Management Table :

Risk Description	Category	Probability	Impact (Severity)	Mitigation Strategy
Job API Downtime – External job APIs (LinkedIn, RapidAPI, etc.) may be unavailable or exceed rate limits, disrupting job search functionality.	Technical	Medium	High	Implement API status monitoring, caching, and retry mechanisms. Maintain backup APIs and modularize integrations for quick updates.
Resume Data Breach – Unauthorized access or leakage of uploaded resumes and personal details.	External	Low	Critical	Encrypt stored data (AES-256), enforce HTTPS/TLS, apply JWT authentication, and restrict file access based on user roles.
AI Inaccuracy (Skill Gap Analysis) – OpenAI API might return irrelevant, outdated, or biased suggestions, impacting user trust.	Technical	Medium	High	Validate AI responses with post-processing checks, improve prompt tuning, and incorporate human feedback loops for retraining.
Scope Creep (Adding Too Many Extra Features) – Expanding features beyond sprint goals may delay timelines or destabilize the system.	Organizational	Medium	Medium	Define sprint scope strictly, conduct change-control reviews, and lock new features after sprint kickoff.

Database Downtime or Data Loss – PostgreSQL crashes, failed migrations, or connection errors, causing loss of user data.	Technical	Medium	High	Automate backups, use transaction-based migrations, test schema in staging before deployment, and monitor DB uptime.
Interview Scheduling Conflicts – Overlapping or incorrect interview slots due to time-zone or calendar synchronization issues.	Technical	Medium	Medium	Validate time zones, detect slot conflicts, provide user confirmation before scheduling, and allow rescheduling options.
Version Control Conflicts – Merge conflicts or incorrect branch management in GitHub may overwrite or lose code updates.	Organizational	Medium	High	Enforce branching strategy (feature → dev → main), mandate pull-request reviews, and conduct frequent merges with CI checks.

Risk Exposure Table:

SL NO	Risk (from Proposal)	Probability (%)	Impact (\$)	Expected Value (RE = Probability × Impact)
1	Job API Downtime	40	\$2,000	\$800
2	Resume Data Breach	15	\$5,000	\$750
3	AI Inaccuracy (Skill Gap Analysis)	35	\$2,500	\$875

4	Scope Creep (Extra Features)	30	\$1,500	\$450
5	Database Downtime or Data Loss	30	\$2,000	\$600
6	Interview Scheduling Conflicts	25	\$1,000	\$250
7	Version Control Conflicts	40	\$1,800	\$720

Quality Factors and Quality Plan:

Quality Factors:

- **Functionality:** The application must allow users to search jobs, upload resumes, manage profiles, track applications, receive notifications, and use AI-based skill gap analysis without errors.
- **Usability:** The UI/UX should be simple, intuitive, and responsive across devices to ensure smooth navigation for job seekers.
- **Reliability:** The system must remain available with minimal downtime, especially when users are applying for jobs or scheduling interviews.
- **Performance:** APIs should respond quickly, with low latency for job searches and AI recommendations, and pages should load quickly.
- **Security:** Resumes and user data must be securely stored, transmitted via HTTPS, and protected against threats like SQL injection, XSS, and unauthorized access.
- **Accuracy:** AI skill gap analysis should provide relevant, meaningful suggestions and improve over time with feedback.

Quality Plan:

- **Regular Testing:** Perform unit testing and integration testing after each sprint.
- **Usability Testing:** Conduct beta testing with real students/job seekers to ensure ease of navigation and effectiveness of AI suggestions.
- **Code Reviews & CI/CD:** Enforce peer code reviews and use GitHub Actions for continuous integration, automated builds, and bug tracking.
- **Load & Stress Testing:** Simulate multiple concurrent users to validate system performance during peak job search or resume uploads.
- **Security Measures:** Follow secure coding practices by encrypting resumes, applying JWT authentication, and conducting regular vulnerability scans.
- **AI Validation:** Continuously validate AI skill-gap results with sample resumes and job descriptions and refine based on feedback.

Design Methods:

System architecture – Model View Controller

The **Career Advisor System** is designed using the **Model–View–Controller (MVC) architecture**, which separates the application into three main components to ensure a structured, scalable, and maintainable system.

The **Model** manages the data and business logic, implemented using **PostgreSQL** with **Prisma ORM** to handle user profiles, resumes, job listings, and application records.

The **View** represents the user interface, built using **React**, **Vite**, and **Tailwind CSS**, providing an interactive and responsive experience for features such as login, resume upload, and job tracking.

The **Controller** acts as the intermediary, managing user requests, processing logic, and updating both the Model and View. It is developed using **Node.js** and **Express.js**, incorporating **Zod** for data validation, **JWT** for authentication, and integration with a **FastAPI microservice** for resume analysis and skill gap recommendations.

This MVC structure ensures clear separation of concerns, enhances team collaboration, and allows easy scalability for future modules like interview scheduling.

- Frontend: Next.js + React for a dynamic and responsive user interface.
- Backend: Node.js/FastAPI implementing modular MVC pattern.
- Database: PostgreSQL for storing normalized relational data.
- Background Tasks: Resume parsing and alert scheduling.

Step-by-Step Implementation Using Diagrams

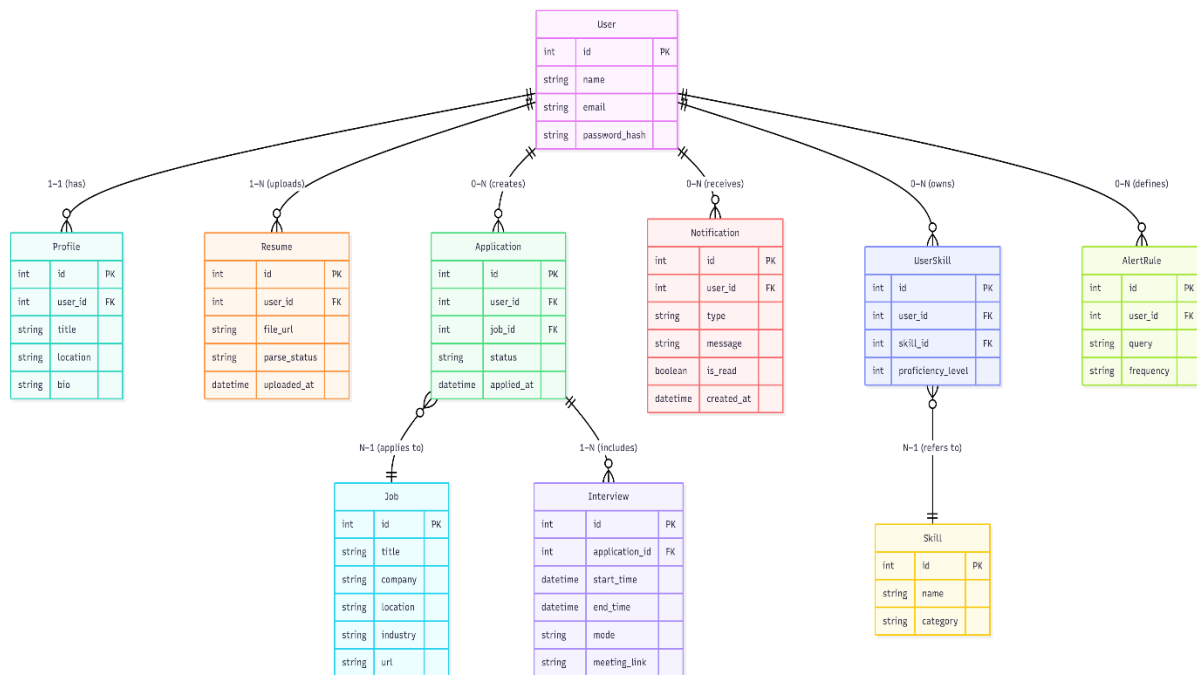
Step 1: Define Database Schema – ER Diagram

Goal: Establish the foundation of the system using a relational database. The ER diagram defines all entities and their relationships.

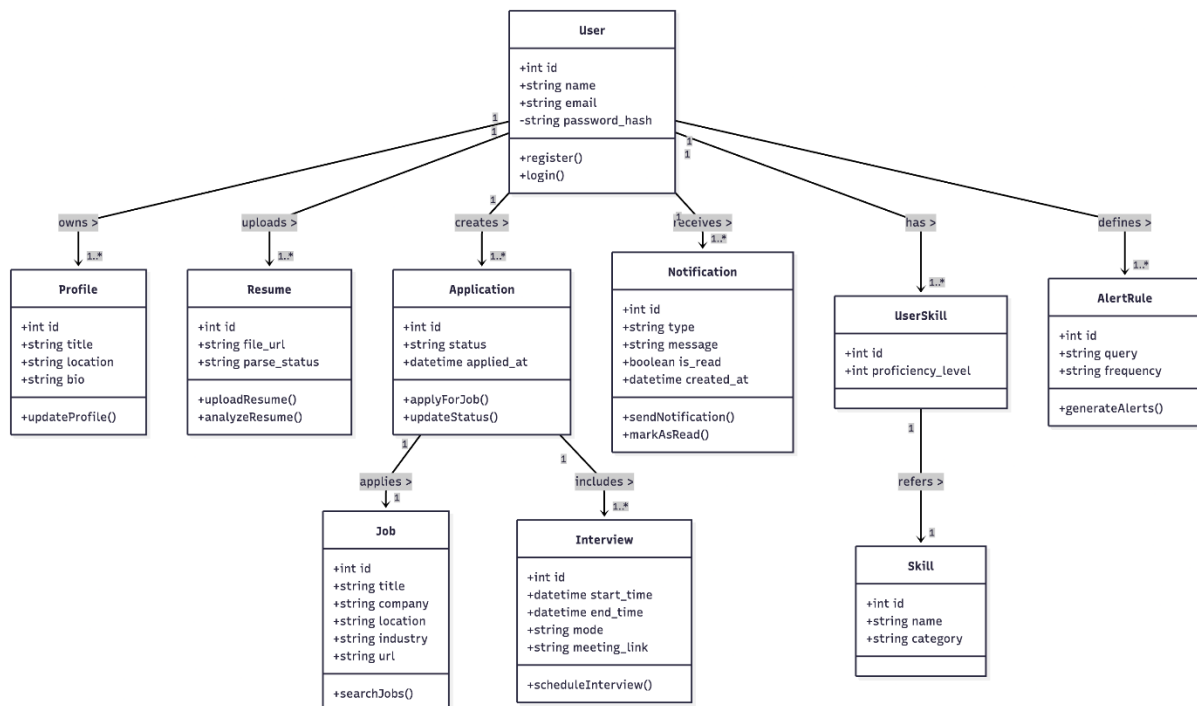
- **Entities:** User, Profile, Resume, Job, Application, Interview, Skill, UserSkill, Notification, and AlertRule.
- **Relationships:**
 - User 1–1 Profile, User 1–N Resume, User 1–N Application, Application N–1 Job, Application 1–N Interview.
- **Normalization:** Prevents redundancy and ensures integrity.

This schema supports all major system functionalities like profile management, job application, interview scheduling, and alert notification.

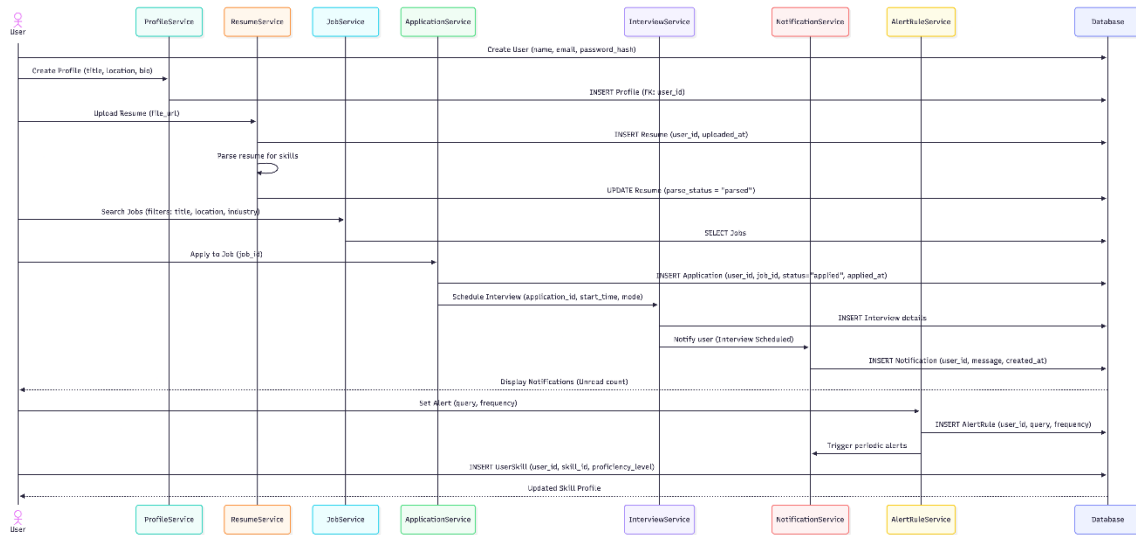
ER Diagram:



Class Diagram:



Sequence Diagram:



Sprint Progress Analysis and Justification

Sprint 1 – On Schedule

The first sprint focused on foundational setup environment configuration, database schema, and basic UI components. The team completed all four tasks on time since this phase was primarily configuration and didn't involve complex dependencies.

Sprint 2 – Completed 1 Day Ahead

Sprint 2 progressed smoothly because several components developed in Sprint 1 were reusable (e.g., shared UI templates and API integration logic). The team was already familiar with the workflow and tools, which improved efficiency and resulted in completing the sprint one day ahead of schedule.

Sprint 3 – Delayed by 1 Day

A minor delay occurred due to **integration challenges between the Resume Analyzer and ATS scoring modules**. During testing, inconsistent PDF parsing caused errors in scoring results,

requiring an additional debugging cycle. The team identified the issue in the PDF parsing library (pdfjs-dist) and fixed it by configuring the worker file path correctly. This caused a **1-day delay**, but the issue was resolved within the same sprint, and testing was completed successfully.

Sprint 4 – Back on Track

The team refocused and optimized their task allocation to recover from the previous sprint's delay. Code reviews and testing were parallelized, and dependencies were better managed. All deliverables were completed as planned, bringing the schedule back on track.

Sprint 5 – Completed 2 Days Ahead

Sprint 5 benefited from the lessons learned in previous sprints. The team had improved CI/CD workflows and better modularized code, allowing for faster integration of the **Job Tracker and Application Filtering** modules. Efficient collaboration and minimal blockers led to finishing the sprint **two days ahead** of the planned schedule.

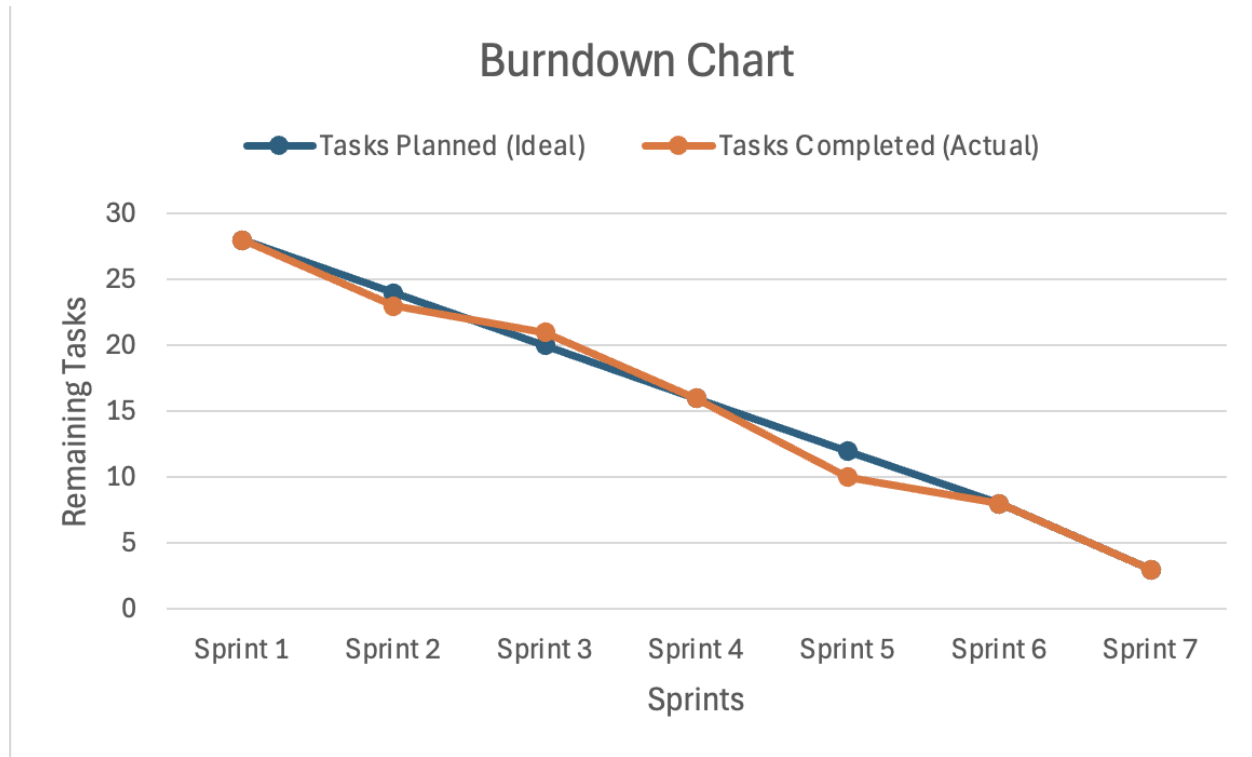
Sprint 6 – Completed on Track

Sprint 6 went smoothly and stayed completely on schedule. The team focused on implementing browser notifications, interview scheduling features. Since the foundation from previous sprints was already set up, development was straightforward and there were no major blockers. Testing activities are planned for the next sprint, so this sprint mainly concentrated on completing all core functionalities on time.

Sprint 7

Sprint 7 experienced a slight delay as additional time was needed to complete comprehensive testing and resolve minor functional bugs. Unexpected issues arose during the deployment phase, particularly with cloud configuration and environment setup.

Burndown Chart:



Implementation:

Hardware Requirements:

Processor: Intel Core i5 / Apple M1 or above

RAM: Minimum 8 GB (Recommended 16 GB for running backend, database, and frontend concurrently)

Storage: At least 50 GB free disk space

System Type: 64-bit operating system

Internet Connection: Required for API calls, deployment, and dependency installation

Software Requirements:

Operating System: Windows 10/11 or macOS

Code Editor: Visual Studio Code

Version Control: Git and GitHub

Runtime Environments:

- **Node.js (v18 or higher)** – for backend server
- **npm** – for package management

Database: PostgreSQL (used with Prisma ORM)

PostgreSQL GUI: pgAdmin / Neon / Supabase

Browser: Google Chrome / Safari (for running frontend UI)

Deployment Platforms: Vercel (frontend), Render or Railway (backend)

Libraries used:

Core Framework:

- **express** – Web framework for building RESTful APIs using Node.js

ORM & Database Access:

- **@prisma/client, prisma** – Object–Relational Mapping tools for PostgreSQL database management

Security:

- **bcryptjs** – For secure password hashing
- **jsonwebtoken** – For authentication using JWT tokens

File Handling:

- **multer** – For uploading resumes and profile images

Validation:

- **zod** – For schema validation and input data checking

Middleware & Configuration:

- **cors** – Enables secure cross-origin resource sharing
- **dotenv** – Manages environment variables securely

Development Tools:

- **nodemon** – Automatically restarts the server during development

Frontend:

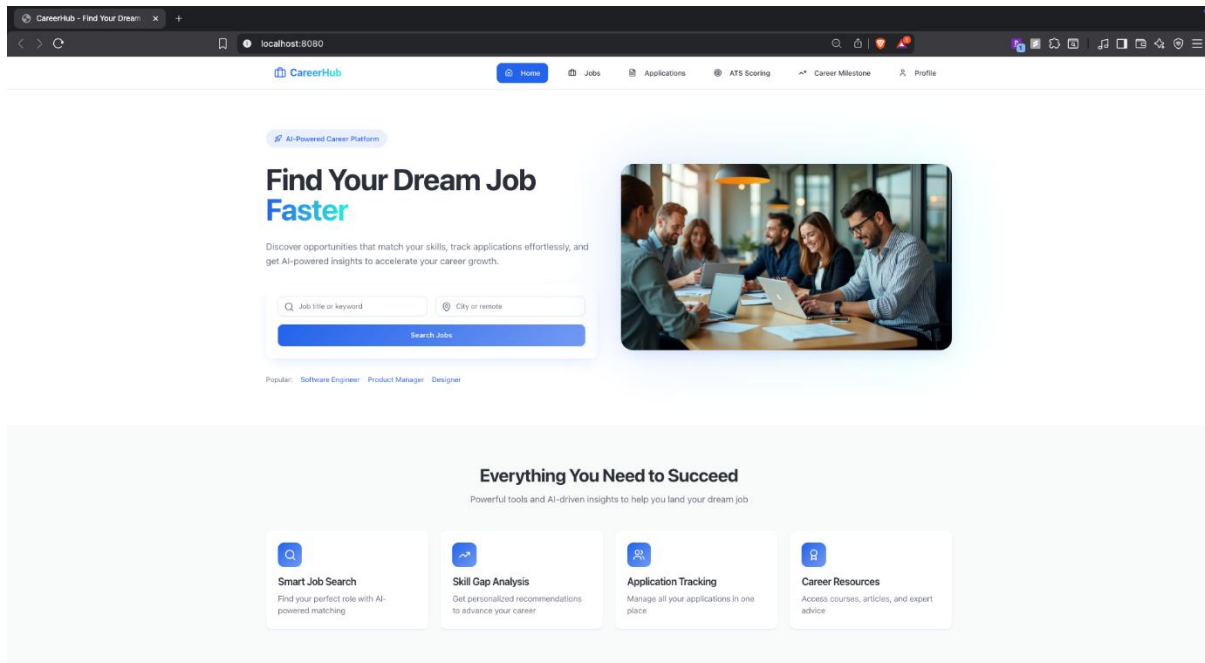
- **React + Vite** for UI rendering

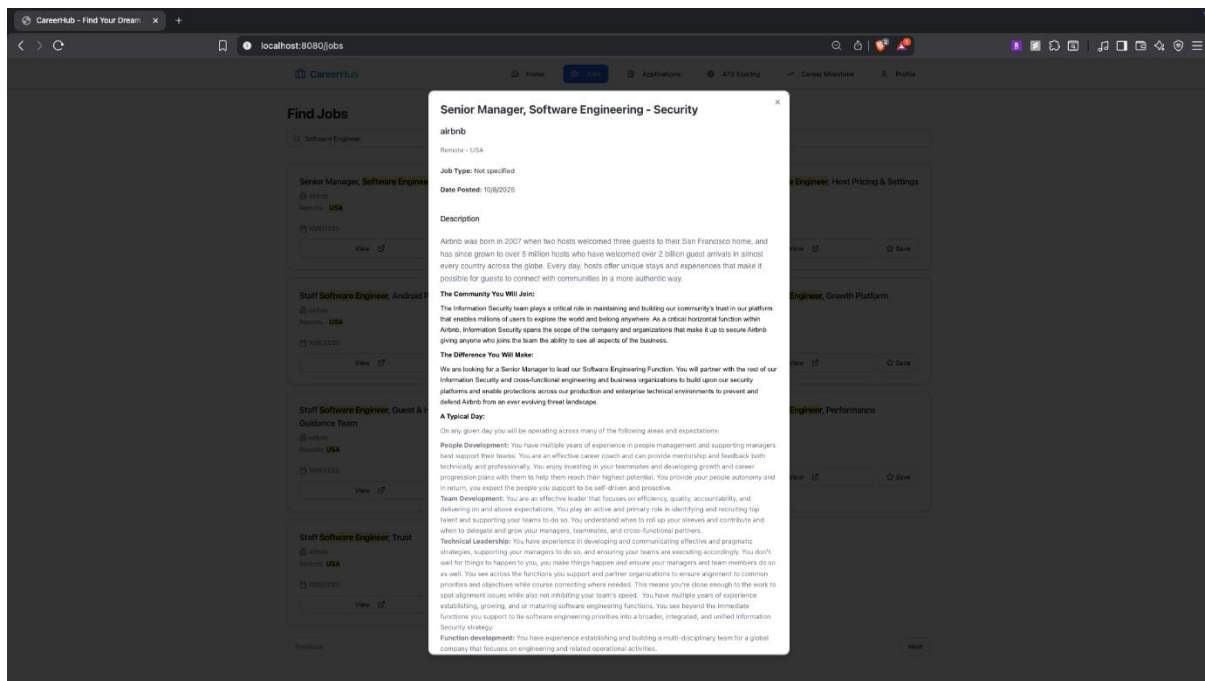
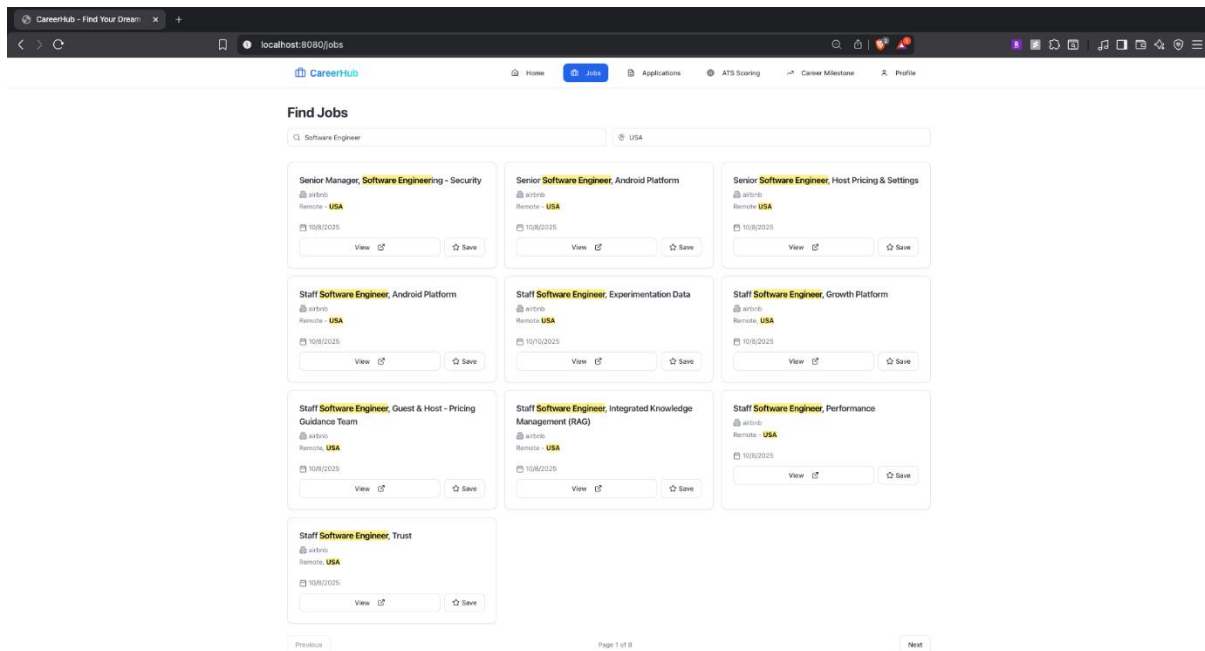
- Tailwind CSS for styling
- shadcn/ui for components
- framer-motion for animations
- axios / fetch API for backend calls

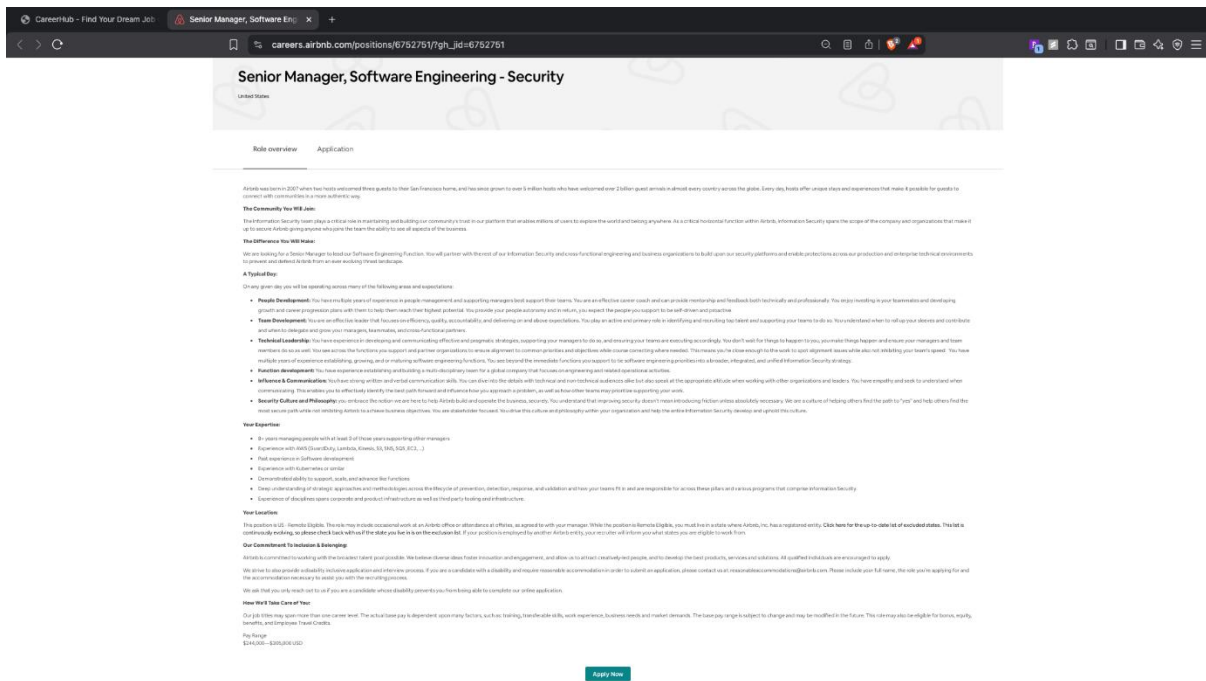
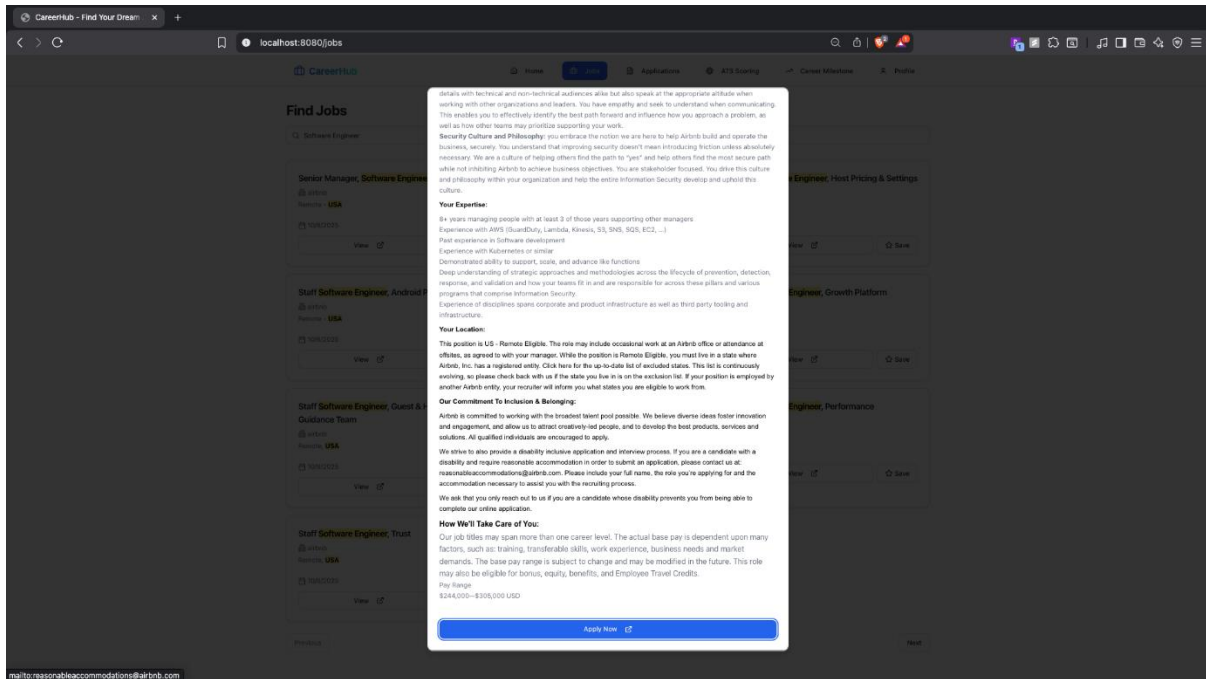
Database:

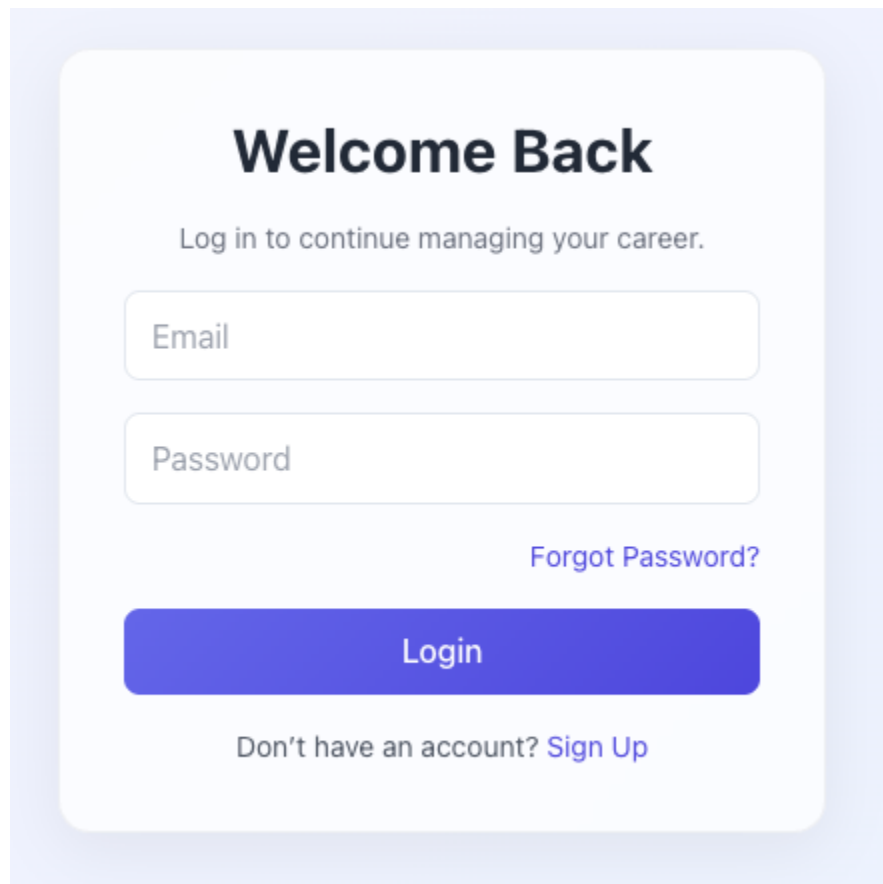
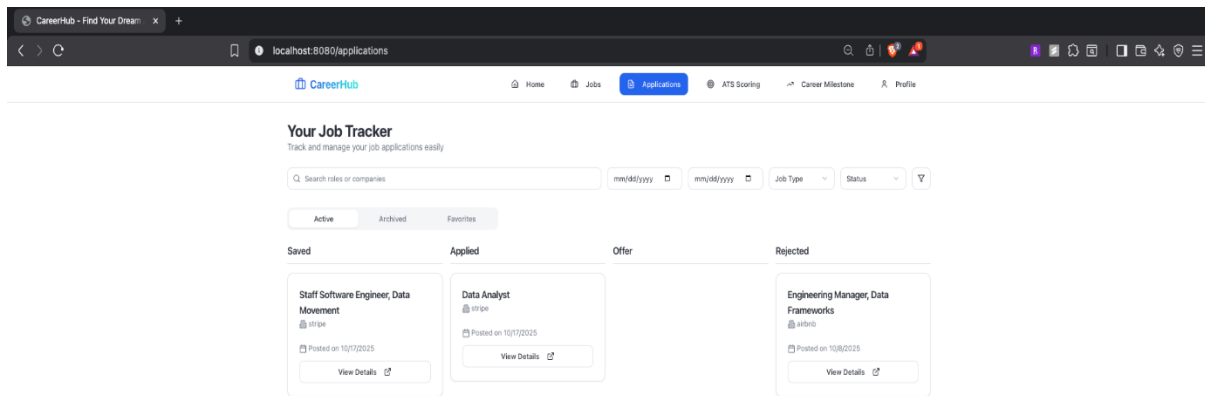
- PostgreSQL – primary data storage
- Prisma ORM – schema management and migrations

User Interface:









Find Your Dream Job Faster

Discover opportunities that match your skills, track applications effortlessly, and get AI-powered insights to accelerate your career growth.

Search Jobs



Popular: [Software Engineer](#) [Product Manager](#) [Designer](#)

Everything You Need to Succeed

Powerful tools and AI-driven insights to help you land your dream job



Smart Job Search

Find your perfect role with AI-powered matching



Skill Gap Analysis

Get personalized recommendations to advance your career



Application Tracking

Manage all your applications in one place



Career Resources

Access courses, articles, and expert advice

Ready to Take the Next Step?

Join thousands of professionals who've found their dream careers through CareerHub

[Browse Jobs](#)[Create Profile](#)

CareerHub

Home

Jobs

Applications

Profile

ATS Scoring

Career Milestone

Interviews

Logout

Software Engineer

USA

Refresh

iOS Software Engineer, Guest & Host

airbnb

Remote, USA

Job Type: Full-time | Work Mode: Remote

11/4/2025

View Details

Save

Senior Software Engineer, Android Platform

airbnb

Remote - USA

Job Type: Full-time | Work Mode: Remote

10/8/2025

View Details

Save

Senior Software Engineer, Host Pricing & Settings

airbnb

Remote USA

Job Type: Full-time | Work Mode: Remote

10/8/2025

View Details

Save

Staff Software Engineer, Android Platform

airbnb

Remote - USA

Job Type: Full-time | Work Mode: Remote

10/8/2025

View Details

Save

Staff Software Engineer, Experimentation Data

airbnb

Remote USA

Job Type: Full-time | Work Mode: Remote

10/10/2025

View Details

Save

Staff Software Engineer - GenAI Systems

airbnb

Remote - USA

Job Type: Full-time | Work Mode: Remote

11/4/2025

View Details

Save

Staff Software Engineer, Growth Platform

airbnb

Remote, USA

Job Type: Full-time | Work Mode: Remote

10/8/2025

View Details

Save

Staff Software Engineer, Host Pricing & Settings

airbnb

Remote, USA

Job Type: Full-time | Work Mode: Remote

11/6/2025

View Details

Save

Staff Software Engineer, Performance

airbnb

Remote - USA

Job Type: Full-time | Work Mode: Remote

10/8/2025

View Details

Save

Previous

Page 1 of 4

Next

CareerHub

Home

Jobs

Applications

Profile

ATS Scoring

Career Milestone

Interviews

Logout

Your Job Tracker

Active

Archived

Favorites

SAVED

APPLIED

OFFER

REJECTED

Staff Software Engineer, Performance

airbnb

View Details

iOS Software Engineer, Guest & Host

airbnb

View Details

Staff Software Engineer, Experimentation Data

airbnb

View Details

Career Milestone

Get personalized course recommendations, articles, and expert AI advice tailored to your career goals

Start Your Journey

Upload your resume and tell us your target job role

Target Job Role

Software Engineer

Upload Resume (PDF)

Choose File Shivesh_Sarava...sume_SDE'.pdf

Shivesh_Saravanan_Resume_SDE'.pdf

Analyze Career Path

Recommended Courses

Full Stack Web Development with React Specialization

Coursera • 4 months

Machine Learning Specialization

Coursera • 3 months

Docker and Kubernetes: The Complete Guide

Udemy • 5.5 hours

Data Structures and Algorithms Specialization

Coursera • 4 months

Cloud Computing Specialization

Coursera • 4 months

Recommended Articles

How to Ace a Software Engineering Interview

LeetCode

10 Tips for Building a Strong Software Engineering Portfolio

Medium

Understanding Microservices Architecture

Martin Fowler

Best Practices for Using Docker in Production

Docker Blog

The Importance of Continuous Integration and Continuous Deployment

DevOps.com

Expert AI Advice

1. Build a strong portfolio showcasing your projects, emphasizing your contributions and technologies used.
2. Network with professionals in the software engineering field on platforms like LinkedIn or GitHub.
3. Stay updated with the latest industry trends and technologies by following influential tech blogs and attending webinars.
4. Practice coding challenges regularly on platforms like LeetCode, HackerRank, or CodeSignal to prepare for technical interviews.
5. Consider obtaining certifications relevant to your skills, such as AWS Certified Solutions Architect or Google Cloud Professional Developer.

1) Upload Resume (PDF)

Upload your resume to analyze against job descriptions



Shivesh_Saravanan_Resume_SDE.pdf

Click to change file

2) Paste Job Description

Copy and paste the job description you're applying for

- Design and Development: Architect, design, and implement scalable software solutions using technologies like Spring Boot, JPA/Hibernate, and Microservices. Develop robust backend features while adhering to best practices. Design complex software systems for scalability, performance, and security. Proficient in designing and implementing microservices architecture using Spring MVC (Java Spring Boot) and Spring Cloud frameworks.
- Security: Implement design patterns and best practices for high-performance backend services, ensuring software security and protection from breaches.
- Quality Assurance: Advocate for quality engineering practices such as unit testing, code reviews, and CI/CD pipelines to deliver reliable and efficient software. Ensure

Additional Information (Optional)

Help us provide more accurate insights

Target Role (optional)

e.g., Data Scientist

Target Location (optional)

e.g., Dallas, TX

Compute Match Score & Draft Cover Letter

ATS Match Score

How well your resume matches the job description

75%

Good Match

Matching Skills

Skills you have that match the job

AWS Microservices Docker CI/CD Agile Python Java Angular

Missing Skills

Skills to develop for this role

Spring Boot JPA/Hibernate Helm Charts API security best practices
Software design patterns Technical documentation standards
7 Years of Relevant Experience Leadership and mentoring experience

AI-Generated Cover Letter

Customized for this job application

Dear Hiring Manager,

I am writing to express my interest in the Sr Principal Software Engineer position at Alcon as advertised. With a strong technical background in software engineering, particularly in web development frameworks and cloud platforms, I am eager to leverage my skills and experience to contribute to Alcon's mission of enhancing lives through innovative eye care solutions. Currently, I am pursuing a Master of Science in Computer Science, which has further deepened my expertise in algorithms, software engineering, and machine learning.

During my tenure as a Software Developer at Holistic MD LLP, I led the engineering of Angular SPAs, focusing on optimizing performance and improving user experience. My hands-on experience with Docker for containerization and CI/CD practices has equipped me with the ability to efficiently manage software deployments and ensure high-quality standards. While I have a solid foundation in microservices architecture, I am excited about the opportunity to advance my knowledge in Spring Boot and JPA/Hibernate within the collaborative environment at Alcon.

I am particularly drawn to this role because of Alcon's commitment to innovation and excellence in patient outcomes. I thrive in fast-paced environments and am passionate about mentoring fellow engineers, fostering a culture of collaboration and continuous improvement. I believe my proactive approach and problem-solving skills align well with the expectations of this role, and I am eager to contribute to the development of scalable and secure software solutions that meet the needs of Alcon's diverse clientele.

Thank you for considering my application. I look forward to the opportunity to discuss how I can contribute to Alcon's goals and help make a meaningful impact in the lives of patients and customers.

Sincerely,
Shivsh Saravanan

Resume Improvement Feedback

Areas to enhance your resume

- Emphasize experience with Spring Boot and JPA/Hibernate since they are critical for the job.
- Highlight leadership and mentoring experience or potential, as this is a senior role.
- Include specific examples of API security practices and design patterns you are familiar with.
- Ensure the resume format is ATS-friendly (e.g., standard fonts, clear headers).
- Consider adding a summary section that outlines your key qualifications and aligns closely with the job description.

Interview Planner

Your Interview Schedule

Click a date to view interviews scheduled that day

November 2025

today

< >

Sun	Mon	Tue	Wed	Thu	Fri	Sat
26	27	28	29	30	31	1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29
30	1	2	3	4	5	6

• 1:57a Staff Software Engi

Interview Details

Thursday, November 13, 2025

Staff Software Engineer, Performance

airbnb • Remote - USA

Online

11/13/2025, 1:57:00 AM – 2:40:00 AM

Join meeting

Edit

Delete

Manual Testing:

Manual testing was carried out for the Career & Job Advisor (CareerHub) application to ensure all key features worked as intended before automation. Major modules such as Login, Signup, Profile, Jobs, Applications, Interview Planner, and ATS Scoring were tested using sample data and API validations. The focus was on verifying user flows, backend API responses, and data consistency across modules. Overall, testing confirmed that core workflows like authentication, profile updates, job tracking, and interview scheduling functioned smoothly and met user requirements.

User Acceptance testing:

Test Case	Test Description	Test Case Instructions (Steps)	Expected Result	Actual Result
TC01	Verify user signup with valid credentials	1. Open /signup. 2. Enter valid name, email, password. 3. Click Sign Up.	User created successfully; redirected to /profile.	Passed – Redirects to /profile.
TC02	Handle invalid signup submission	1. Leave fields empty / invalid email. 2. Submit form.	Validation errors shown; no redirect.	Passed – Errors shown properly.
TC03	Verify user login with correct credentials	1. Enter registered email and password. 2. Click Login.	Dashboard/profile loads successfully.	Passed – JWT stored; dashboard loaded.
TC04	Handle login with invalid credentials	1. Enter invalid password. 2. Click Login.	Error “Invalid credentials” displayed.	Passed – Error handled correctly.
TC05	Forgot password – send OTP	1. Click Forgot Password. 2. Enter valid registered email. 3. Click Send OTP.	OTP email sent successfully.	Passed – OTP received via Nodemailer mock.
TC06	Forgot password – verify OTP and reset password	1. Enter OTP. 2. Enter new password and confirm. 3. Submit.	Password reset successfully; user can log in with new credentials.	Passed – Password updated in DB.
TC07	Save and persist profile details	1. Go to /profile. 2. Edit fields and click Save.	Profile updates saved; confirmation shown.	Passed – Data persisted.

TC08	Upload and view resume file	1. Upload valid PDF/DOCX. 2. Click Submit then View Resume.	Resume uploaded, stored, and viewable.	Passed – Working properly.
TC09	Search and filter jobs by role & location	1. Enter role and location filters. 2. Click Search.	Filtered job list displayed.	Passed – Correct data fetched.
TC10	Handle no job results gracefully	1. Enter unmatched filters.	“No results found” displayed.	Passed.
TC11	Add a new job application	1. Go to /applications. 2. Click Add Application. 3. Fill job details and save.	Application added successfully.	Passed.
TC12	List all job applications	1. Navigate to /applications.	All applications and statuses displayed correctly.	Passed.
TC13	Schedule a new interview	1. Open /interview-planner. 2. Add interview details (job, date/time, mode).	Interview added and displayed on calendar view.	Passed.
TC14	Edit existing interview	1. Edit existing interview. 2. Save changes.	Updated interview appears correctly; no duplication.	Passed after patch.
TC15	AI Resume Analysis	1. Upload resume. 2. Call /ai/analyze Resume.	Returns ATS score + extracted skills.	Passed.
TC16	Skill Gap Analysis	1. Compare resume vs target job description.	Missing/required skills listed in response.	Passed.
TC17	Career Milestone Recommendations	Career milestone -> User role->system analyzes resume	Recommended articles, courses, and AI expert advice displayed.	Passed
TC18	Health Check Route	1. Run GET /health.	Returns {status: "ok"}.	Passed.
TC19	Unknown Route Handling	1. Run GET /invalid.	Returns 404 with JSON error.	Passed.
TC20	End-to-End Integration	Execute full workflow across modules.	All modules integrate seamlessly; no data inconsistency.	Passed – End-to-end verified.

Frontend testing:

Test ID	Test Description	Test Steps (How to Perform)	Expected Result	Actual Result
FE-HOM-01	Home → Job Search	Enter “Job title or keyword” → “City or Remote” → Click Search Jobs	Navigates to /jobs?q=Frontend&loc=Austin; “JOBS PAGE” rendered	Pass
FE-HOM-02	Popular Tags Navigation	Click Software Engineer tag	Navigates to /jobs?q=Software Engineer; page renders	Pass
FE-HOM-03	Create Profile (Guest)	Logged out (no token) → Click Create Profile	Redirects to /signup and shows “SIGNUP PAGE”	Pass
FE-HOM-04	View Profile (Logged in)	localStorage token set → Click View Profile	Redirects to /profile and shows “PROFILE PAGE”	Pass
FE-HOM-05	Browse Jobs CTA	Click Browse Jobs	Navigates to /jobs; “JOBS PAGE” rendered	Pass
FE-LOG-01	Login Success	Enter email + password → Click Login	API called; token stored; redirect to /profile; “PROFILE PAGE” visible	Pass
FE-LOG-02	Login Error Toast	Enter wrong password → Click Login	toast.error("bad"); stays on /login	Pass
FE-LOG-03	Login → Signup Link	Click Sign Up text link	Navigates to /signup; shows “SIGNUP PAGE”	Pass
FE-SGN-01	Signup Success	Fill first/last/email/password → Click Sign Up	Calls /auth/signup; toast.success; redirect → /verify-email	Pass
FE-SGN-02	Signup Error	Mock API failure → Submit form	toast.error; stays on /signup	Pass
FE-SGN-03	Signup → Login	Click Log in text span	Navigates to /login; shows “LOGIN PAGE”	Pass
FE-PRO-01	Profile Load and Render	Mock profile API → Open /profile	“My Profile” + field values (John Doe Email Skills) render	Pass
FE-PRO-02	Add / Remove Skill	Type “TS” in Add Skill → Press Enter → Click × icon	Chip adds then removes from DOM	Pass
FE-PRO-03	Hide Empty AI Sections	Mock empty aiCourses/articles/advice	AI sections not rendered; Save/Upload buttons visible	Pass
FE-APP-01	Load Applications	Mock API → Open Applications page	Shows “Your Job Tracker” + tabs + cards	Pass
FE-APP-02	View Details Dialog	Click View Details	Opens Dialog with Title & Description	Pass
FE-APP-03	Status Change (PATCH)	Open dialog → Change status via Select	PATCH called; dialog stays open	Pass

FE-APP-04	Schedule Interview	Click Schedule Interview → set times → Save	POST to /interviews; handles 409 conflict	Pass
FE-APP-05	Favorite / Archive	Click Favorite , then Archive	PATCH calls made for both actions	Pass
FE-APP-06	Delete Application	Click Delete → confirm mock DELETE	DELETE API called; card removed	Pass
FE-INT-01	Interview Planner Load	Open page → mock API data load	Renders calendar + “Your Interview Schedule”	Pass
FE-INT-02	Reminder Scheduler	Wait for load → verify apiFetch triggered	Hook runs without error	Pass
FE-ATS-01	Resume Upload Validation	Upload .txt → reject → upload .pdf → accept	Only PDF accepted; stored file = r.pdf	Pass
FE-ATS-02	Compute ATS Results	Upload resume + paste JD → Click Compute Match Score	Renders Cover Letter + skills + feedback areas	Pass
FE-MIL-01	Run Career Analysis	Provide Role + Resume → Click Analyze	Shows Courses, Articles, Advice sections	Pass
FE-MIL-02	Analyze Button State	Try clicking Analyze w/o inputs → add role only → add resume	Disabled until both given	Pass
FE-ERR-01	404 Fallback	Navigate to unknown route	Shows “404” + “Return to Home” link (/)	Pass

Backend Testing:

Test ID	Test Description	Test Type	Test Steps (How to Perform)	Expected Result	Actual Result
T01	Verify user signup flow with OTP email	Integration	Mock POST /api/auth/signup; verify prisma.user.create + sendVerificationEmailcalls	200/201 Created + OTP email sent	Passed
T02	Invalid password during login	Unit	Mock invalid password → bcrypt compare fails	401 Unauthorized	Passed
T03	Forgot password triggers reset email	Unit	Mock valid email; verify reset email mock	200 OK + email sent	Passed
T04	OTP verification with wrong code	Unit	Mock invalid OTP request	400 Bad Request	Passed
T05	Full signup → verify → login flow	Integration	Chain signup → verify → login using mocked Prisma + bcrypt	Token issued + 200 OK	Passed
T06	Validate missing JWT token	Unit	Call middleware w/out Authorization header	401 Unauthorized	Passed
T07	Validate valid JWT token	Integration	Mock JWT.verify → sets req.user → calls next()	Request passes to next handler	Passed
T08	Schema validation – invalid body	Unit	Pass invalid body to middleware	400 Bad Request	Passed
T09	Schema validation – valid body	Unit	Pass valid body → next()called	Request continues	Passed
T10	Error handler uses custom status	Unit	Throw error with status 418	Returns 418 and JSON error	Passed
T11	Default error handler (500)	Unit	Throw generic error (no status)	Returns 500 Internal Error	Passed
T12	Upsert user profile	Integration	Mock PUT /api/profilewith user data	200 JSON payload (profile updated)	Passed
T13	Reject invalid resume file type	Unit	Upload .exe (non-PDF)	400 Invalid file type	Passed
T14	Replace old resume with new PDF	Integration	Upload PDF → expect old delete/new save	200 Success JSON	Passed
T15	List all applications	Unit	Mock GET /api/applications	{applications: [...]} returned	Passed
T16	Create new application	Integration	Mock POST /api/applications with payload	201 Created	Passed
T17	Update status with invalid value	Unit	Mock invalid status → catch in next()	Error handled gracefully	Passed
T18	Unauthorized application update	Integration	Simulate non-owner update	403 Forbidden / handled	Passed

T19	Missing application update	Unit	Update non-existing record	404 or next(e)called	Passed
T20	Delete application (non-owner)	Integration	Delete as other user	403 / 404 error	Passed
T21	Delete application (owner)	Integration	Delete as owner (user.id matches)	204 No Content or 200	Passed
T22	Search jobs success path	Integration	Mock provider API → aggregate results	JSON with jobs array	Passed
T23	Provider failure fallback	Unit	Simulate failed fetch → status 503	{jobs: []}returned	Passed
T24	Health endpoint ping	Integration	GET /api/ping	{status:"ok"}returned	Passed
T25	Not found route	Integration	GET unknown route	{error:"Not found"}returned	Passed

Automation Testing:

Automation testing in the CareerHub – AI-Powered Career & Job Advisor application ensures that core features such as login, signup, job search, profile management, and resume upload work correctly after every update. Using Selenium WebDriver with Mocha, real user actions are simulated to verify UI flow, navigation, and functionality across modules. Automated HTML reports (Mochawesome) provide quick insights into test results, helping maintain quality, speed, and reliability throughout continuous development.

selenium-e2e

Tuesday, November 11, 2025 9:13pm

Show Passed

Show Failed

Show Pending

Show Skipped

Show Hooks

Failed

Basic: a11y roles (presence only)

Applications: change status

Applications smoke test

ATS: upload confirms

ATS upload smoke test

Auth: guarded route redirects

Auth: login happy path

Empty state: Applications

Home smoke test

Jobs: filter behavior

Jobs smoke test

Layout (header/footer)

Login smoke test (passes without backen...

Navigation links

NotFound smoke test

Profile: save

Profile smoke test

Route smoke (render app shell)

CareerHub E2E Sanity (ESM)

Users/shivesh/Documents/cja%20.20.00%E2%80%AFAM/selenium-e2e/artifacts/selenium-re... | 🔍 🚀

🕒 1m 14.7s 📄 22 🗑️ 29 ✅ 29 ❌ 0

Basic: a11y roles (presence only) ^

Applications: change status ^
spec.js

Applications smoke test ^

ATS: upload confirms ^

ATS upload smoke test ^

Auth: guarded route redirects ^

Auth: login happy path ^

Empty state: Applications ^

Home smoke test ^

Jobs: filter behavior ^

Jobs smoke test ^

Layout (header/footer) ^

Login smoke test (passes without backen... ^

Navigation links ^

NotFound smoke test ^

Profile: save ^

Profile smoke test ^

Route smoke (render app shell) ^

CareerHub E2E Sanity (ESM) ^

The Career & Job Advisor (CareerHub) application has been designed with scalability and modularity in mind, allowing for continuous enhancements post-deployment. Future maintenance will primarily focus on expanding platform accessibility, strengthening integrations, improving system management, optimizing deployment, and ensuring data security. The following key improvements are planned for future versions:

Develop a cross-platform mobile app using React Native or Flutter to enhance accessibility and allow users to manage their career profiles, applications, and interviews directly from their phones.

Integration with External Platforms:

Connect the system with job portals such as LinkedIn, Indeed, and Glassdoor to fetch real-time job data and synchronize user career profiles for improved job recommendations

Admin & Recruiter Portal:

Introduce an admin and recruiter dashboard for managing job listings, monitoring user engagement, and overseeing AI-generated recommendations and analytics.

Cloud Optimization & CI/CD:

Deploy the system on a scalable cloud infrastructure with Kubernetes orchestration and implement CI/CD pipelines for efficient updates and reduced downtime.

Security & Compliance Updates:

Continuously enhance application security through encryption, vulnerability testing, and ensure compliance with GDPR and modern data protection standards.

Roles and Responsibilities:

Team Members	Front-End Responsibilities	Back-End Responsibilities	Testing Responsibilities
Shivesh Saravanan	Designed and implemented UI for Login, Signup, and Profile Management pages using React and Tailwind CSS. Worked on responsive layouts and component reusability.	Developed Authentication and Profile Management APIs using Express and Prisma with PostgreSQL integration.	Performed unit testing for authentication and profile APIs, integration testing for user workflows, and Selenium testing for UI validation.
Nantha Kumar Ashok Anand	Developed interactive UI for Job Search and Application Tracker modules with filtering and dynamic search features.	Implemented Job CRUD operations, Application Management, and AI Scoring integration in the backend.	Conducted unit tests for job modules, integration testing for job-application flow, and Selenium testing for end-to-end functionality.

Varun Sambath Kumar	Designed UI for Interview Planner and Career Milestone modules with calendar components and recommendation cards.	Built Interview Scheduling and Career Recommendation APIs integrated with AI endpoints for personalized suggestions.	Executed unit tests for recommendation and scheduling APIs, integration testing for career milestones, and Selenium automation for the overall workflow.
---------------------	---	--	--

References:

React Documentation – Meta Platforms, Inc. Retrieved from <https://react.dev/>

Next.js Framework Guide – Vercel Inc. Retrieved from <https://nextjs.org/docs>

PostgreSQL Reference – PostgreSQL Global Development Group. Retrieved from <https://www.postgresql.org/docs/>

Tailwind CSS Docs – Tailwind Labs Inc. Retrieved from <https://tailwindcss.com/docs>

OpenAI API Reference – OpenAI. Retrieved from <https://platform.openai.com/docs>

Lever API Documentation – Lever, Inc. Retrieved from <https://developers.lever.co>

Greenhouse API Documentation – Greenhouse Software, Inc. Retrieved from <https://developers.greenhouse.io>

Express.js Documentation – OpenJS Foundation. Retrieved from <https://expressjs.com/>

Prisma ORM Docs – Prisma Data, Inc. Retrieved from <https://www.prisma.io/docs>

Selenium WebDriver Docs – SeleniumHQ Project. Retrieved from <https://www.selenium.dev/documentation/webdriver/>